

操作系统并发专题：信号量与 PV 操作方法论

从执行交错到 408 经典同步题的可执行建模方法

408 专题手册系列 · OS 并发模块

定位：PV 不是口诀，而是约束映射与语义建模

信号量/PV 题不从套模板开始，而从共享状态、等待条件和不变量推导信号量：

执行流 → 交错 → 共享状态 → 不变量 → 约束 → 信号量/PV → 进展性与代价

面对陌生题，先回答：

谁在动？共享什么？谁必须等谁？哪些步骤不可交错？初始系统到底有多少“许可”？

在本手册中，我们严格区分三个核心概念：

- **许可计数 (Permit Count)**：信号量整型变量 S 的当前可用数值（如 $S = 1$ 表示 1 个互斥进入名额， $S = N$ 表示 N 个缓冲区空位额度）。
- **约束映射 (Constraint Mapping)**：将现实物理/业务规则推演翻译为 PV 逻辑的思考建模过程。
- **PV 伪代码 (PV Pseudocode)**：答题卡上最终写出的 $P(S)$ / $V(S)$ 程序代码片段。

内容边界

这里定义信号量的许可语义、PV 建模流程、经典同步模式和正确性检查。进程状态转换由进程与调度专题定义；本专题只使用 Running→Blocked、Blocked→Ready 等状态结果。

目录

1 第零章：为什么会有并发问题

1.1 并发真正研究的对象是“执行轨迹集合”

在并发编程中，无法只看静态代码，必须分析动态运行时的指令执行序列：

- 多线程指令交错 (Interleaving)：并发运行时，CPU 调度器可以在任意时刻切换线程。线程 T_1 和线程 T_2 的指令在时间轴上交叉拼合，称为“指令交错”。
- 执行轨迹 (Execution Trajectory / Trace, 记作 σ)：把一次运行中真实发生的全局指令顺序记录下来，构成的具体流水账叫“执行轨迹”。
- 执行轨迹集合 (Set of Trajectories, 记作 Σ)：由于 CPU 调度的随机性，所有可能产生的交错流水账构成集合 $\Sigma = \{\sigma_1, \sigma_2, \dots\}$ 。
- 并发控制的目标：单次运行成功仅代表集合 Σ 中的某一条轨迹 σ 正确。并发控制的本质，是通过 P/V 操作修剪这个轨迹集合 Σ ，剔除所有引发数据竞争和死锁的错误轨迹。

设有两个执行流 T_1, T_2 共同操作共享状态 S 。某一条具体交错轨迹表示为：

$$\sigma = T_1^1, T_2^1, T_1^2, T_2^2, \dots$$

并发程序正确性定义为：对所有允许交错轨迹，系统状态都必须满足安全不变量 I ：

$$\forall \sigma \in \text{Allowed Interleavings}, \quad I(S_\sigma) = \text{true}$$

符号 / 术语	严谨定义与直观含义解析
σ (Sigma)	某一条具体的执行轨迹。一次运行中 CPU 按时间顺序执行的指令步骤流水账。
T_i^j	进程/线程 T_i 的第 j 个原子步骤。例如 T_1^1 表示线程 1 执行的第一条汇编指令。
S_σ	沿着轨迹 σ 执行完毕后的共享状态。如变量 <code>counter</code> 的最终值。
$I(S)$	系统不变量 (Invariant)。无论何时都绝对不能被破坏的安全业务规则（如 $0 \leq count \leq N$ ）。
$\forall \sigma \in \text{Allowed Interleavings}$	对允许轨迹集合中的任意一条轨迹。要求对所有合法交错组合全部成立。

并发篇第一原则

并发不是“执行顺序不确定”这么简单，而是：程序员不能假定调度器会在方便的位置切换；高层语句又可能由多条底层步骤组成。因此正确性必须对所有允许交错成立。

1.2 综合题：一条 `counter++` 的一生

程序员看到：

```
counter++
```

机器可能执行：

```
load counter
```

```
add 1
store counter
```

若初值为 5，两个线程可能发生：

```
T1: load 5
T2: load 5
T1: add  -> 6
T2: add  -> 6
T1: store 6
T2: store 6
```

最终得到 6 而不是 7，称为**丢失更新 (Lost Update)**。

这条最小反例依次推出：

Shared State → Interleaving → Race → Atomicity → Critical Section → Mutual Exclusion

1.3 同步不等于互斥

并发控制中最基本的两个关系必须分开：

关系类型	本质驱动问题	典型工具与直观表现
互斥 (Mutual Exclusion)	哪些关键指令/状态更新不能彼此交错？	Mutex、二元信号量 (Initial=1)、临界区
条件/顺序同步 (Synchronization)	什么条件满足后才能继续？谁必须先于谁？	计数信号量 (Initial=N/0)、事件信号量、条件变量

考试识别

看到“共享变量、共享缓冲区、文件写入、计数器”等，先找**不允许交错的状态更新**；看到“有产品才能消费、A 完成后 B 才能做、资源数量有限”等，先找**等待条件/前驱关系**。很多 PV 大题是两者叠加。

2 信号量到底是什么：把并发约束映射为许可计数

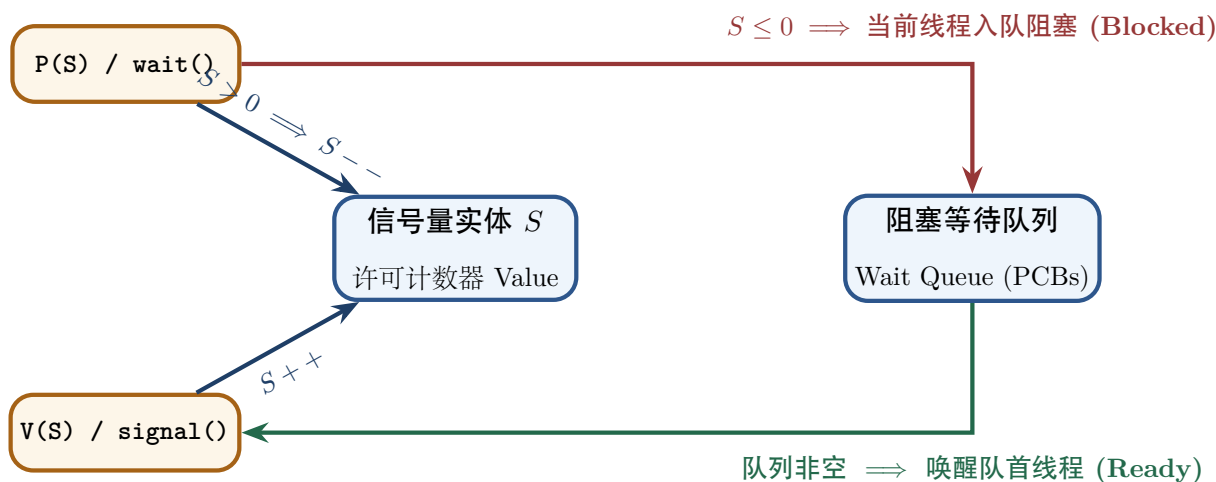
2.1 最稳定的理解：Permit Counter + Waiting Queue

信号量可以理解为：

Semaphore = Permit Counter + Atomic Wait/Wake Mechanism

它不是“神秘整数”，而是把某种现实业务规则映射为具体可消费的许可数量 (Permit Count)：

- $S = 1$ ：当前有 1 个进入资格，可用于单资源互斥；
- $S = N$ ：当前有 N 个同类资源或容量许可；
- $S = 0$ ：某个必须等待的事件尚未发生，或当前没有可消费资源。



初值不是背出来的

问：系统初始时到底已经拥有多少份可以立即被消费的事实/资源/空位/许可？这个数量就是计数信号量初值的来源。

2.2 P/V 的两种常见教材记法

不同教材会采用不同内部表示。做题时首先服从题目定义。

记法 A：可出现负值的经典教学模型

$P(S): S \leftarrow S - 1; \quad S < 0 \Rightarrow$ 当前进程阻塞入队,
 $V(S): S \leftarrow S + 1; \quad S \leq 0 \Rightarrow$ 从等待队列唤醒一个进程.

在这种模型中，负值的绝对值常可解释为等待者数量。

记法 B：计数保持非负的抽象模型

- **wait/P**: 若有许可则原子地消费一个；若无许可则阻塞；
- **post/V**: 增加一个许可；若有等待者则允许其中一个继续竞争运行。

两者语义一致：**P** 可能阻塞，**V** 可能唤醒，且 **P/V** 自身必须原子。

2.3 P/V 为什么必须是原子操作

如果 $P(S)$ 自己可以被拆成“读 S 、判断、减一、入队”等可交错步骤，那么两个执行流可能同时认为自己拿到了同一份许可。信号量的正确性因此必须最终依赖更底层的原子机制，如 **test-and-set**、**compare-and-swap**、**fetch-and-add** 或内核的受保护临界区。

抽象层次

硬件原子能力 $\rightarrow$ 锁/阻塞原语 $\rightarrow$ 信号量/条件变量 $\rightarrow$ 并发算法

408 通常从信号量层开始做题，但理解下面一层能解释“为什么 **P/V** 本身可以被信任”。

2.4 二元信号量与 Mutex：考试等效，语义并非完全相同

408 里常说“二元信号量可实现互斥”，这是正确的考试模型。但严格设计上：

- mutex 强调**所有权 (ownership)**：谁加锁，通常由谁解锁；
- semaphore 强调**许可数量**：一个执行流 P，另一个执行流 V 完全可以有合理语义，例如事件同步。

因此应记：

Binary Semaphore can implement mutual exclusion, but is not conceptually identical to a mutex.

3 PV 大题统一建模算法：先建模，再填 P/V

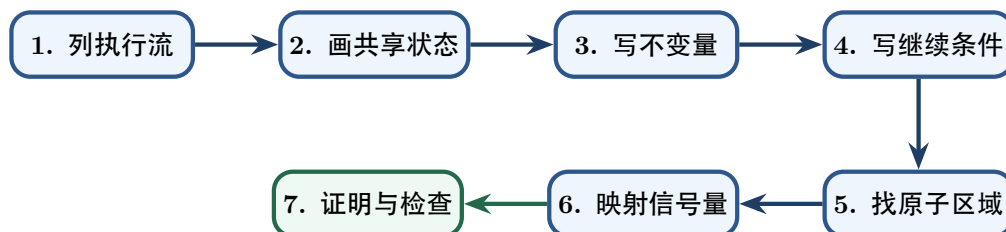
3.1 旧式做法的问题

“找互斥、找同步、塞 P/V”很实用，但遇到复杂变式时容易机械。更稳定的方法是从共享状态和不变量出发。

3.2 七步法

PV 通用七步法

1. 列执行流：有哪几类进程/线程？每类反复执行什么业务动作？
2. 画共享状态：共享缓冲区、计数器、资源、标志位、队列分别是什么？
3. 写不变量：哪些关系永远不能被破坏？如 $0 \leq count \leq N$ 。
4. 写继续条件：每类执行流在什么谓词成立时才能继续？
5. 找原子区域：哪些多步修改必须表现得像一个整体？
6. 把约束映射成信号量：互斥许可、资源数量、事件前驱分别需要什么信号量及初值？
7. 证明与检查：Safety、Deadlock、Liveness、Starvation、锁粒度和 P/V 完备性。



3.3 “先同步 P，再互斥 P”不是宇宙定律

在标准有界缓冲区中：

```

P(empty);
P(mutex);
... put item ...
  
```

```
V(mutex);
V(full);
```

是正确经典模板。

错误写法：

```
P(mutex);
P(empty); // 若 empty==0, 则持有 mutex 睡眠
```

可能死锁，因为只有消费者进入同一临界区才能制造新的 empty，但 mutex 已被生产者占有。

真正应记的原则

不要机械背“所有题都先同步后互斥”。真正规律是：

不要在持有某个资源/锁时，等待一个只能由需要该资源/锁的其他执行流才能制造的条件。

复杂题应画等待依赖，而不是套固定顺序。

3.4 V 的顺序也不要绝对化

标准生产者-消费者通常写：

$$V(mutex) \rightarrow V(full)$$

这样先释放临界区，再通知消费者，避免刚唤醒的消费者马上又阻塞在 mutex 上。某些简单模型里交换两个 V 不一定破坏 Safety，但会改变调度/性能，复杂协议里还可能影响语义。因此不要记成“V 顺序永远无所谓”。

4 模式一：生产者-消费者——容量计数 + 互斥更新

4.1 问题模型与不变量

容量为 N 的有界缓冲区：

$$0 \leq count \leq N$$

生产者继续条件：

$$count < N$$

消费者继续条件：

$$count > 0$$

同时，插入/删除和缓冲区元数据更新不能被多个执行流破坏。

4.2 信号量设计

信号量对象	初值 (Initial)	现实含义与许可类型
empty	N	可立即消费的空位许可数量（同步信号量）
full	0	可立即消费的产品许可数量（同步信号量）
mutex	1	进入缓冲区结构更新临界区的唯一许可（互斥信号量）

关键守恒关系可辅助检查：

$$empty + full = N$$

（在经典抽象模型、忽略正在 P/V 过渡的瞬间进行逻辑理解。）

4.3 标准模板

```
semaphore mutex = 1;
semaphore empty = N;
semaphore full  = 0;

producer() {
    while (true) {
        produce_item();
        P(empty);
        P(mutex);
        put_item();
        V(mutex);
        V(full);
    }
}

consumer() {
    while (true) {
        P(full);
        P(mutex);
        get_item();
        V(mutex);
        V(empty);
        consume_item();
    }
}
```

4.4 为什么生产和消费要放在锁外

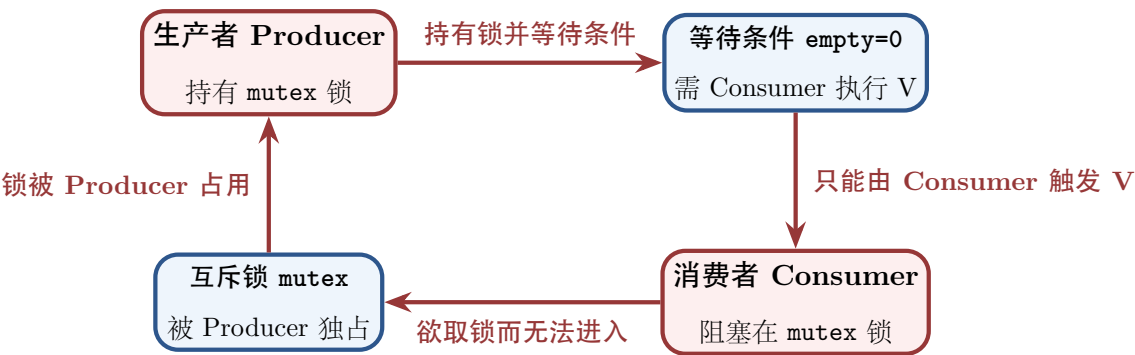
produce_item() 与 consume_item() 若只使用私有数据，不应放入缓冲区 mutex 临界区。否则虽然 Safety 仍可正确，却会把长时间工作串行化，降低并发度。

4.5 经典死锁轨迹

缓冲区已满，若生产者先：

```
P(mutex);    // 成功
P(empty);    // empty=0 -> Blocked
```

则生产者睡眠时仍持有 mutex。消费者即使 Ready/Running，也会阻塞在 P(mutex)，无法取出产品并执行 V(empty)。形成：



本质是等待环。

4.6 典型变式

- 单/多生产者、单/多消费者：基本模板不变；
- 消费一个 A 和一个 B：分别建立 fullA、fullB，消费者要取得两类许可；
- 多类资源同时获取：开始接近哲学家/多资源分配问题，需要检查循环等待和持有后等待；
- 批量生产/消费 k 个：不能只机械连续执行 k 次 P；要明确题目是否要求“整批原子获得”，以及容量/公平性要求。

5 模式二：读者-写者——群体准入 + 元数据保护

5.1 约束结构

约束：

- 读-读允许；
- 读-写互斥；
- 写-写互斥。

它不是普通“一把锁保护所有人”，而是**群体准入 (group admission)**：多个读者构成一个共享访问群体，第一个读者负责关闭写者入口，最后一个读者负责重新开放。

5.2 读者优先模板

```
semaphore rw_mutex = 1; // 数据区读写锁
semaphore r_mutex = 1; // 保护 readcount
int readcount = 0;

reader() {
    P(r_mutex);
    if (readcount == 0)
        P(rw_mutex);    // 第一个读者封锁写者
    readcount++;
    V(r_mutex);

    read_data();

    P(r_mutex);
    readcount--;
    if (readcount == 0)
        V(rw_mutex);    // 最后一个读者释放写者
    V(r_mutex);
}

writer() {
    P(rw_mutex);
    write_data();
    V(rw_mutex);
}
```

5.3 为什么 readcount 本身必须保护

`readcount++`、判断“是否第一个读者”、`readcount--`、判断“是否最后一个读者”共同决定群体状态。如果它们交错，就可能让多个读者都误以为自己是第一/最后一个，破坏准入协议。

读者-写者真正训练的东西

它训练的不是“多加一把锁”，而是：**共享数据和管理共享数据的元数据都可能是临界状态**。元数据（如计数器、索引、状态位）常常比业务数据本身更容易被忽略。

5.4 公平性是独立策略维度

上述方案是读者优先：只要读者不断到来，写者可能长期无法获得 `rw_mutex`，即**写者饥饿**。

可以再增加“入口闸门”和写者计数实现写者优先：

```
int read_count = 0;
int write_count = 0;
semaphore r_mutex = 1;
semaphore w_mutex = 1;
```

```

semaphore rw_mutex = 1;
semaphore read_try = 1;

writer() {
    P(w_mutex);
    write_count++;
    if (write_count == 1) P(read_try);
    V(w_mutex);

    P(rw_mutex);
    write_database();
    V(rw_mutex);

    P(w_mutex);
    write_count--;
    if (write_count == 0) V(read_try);
    V(w_mutex);
}

reader() {
    P(read_try);
    P(r_mutex);
    read_count++;
    if (read_count == 1) P(rw_mutex);
    V(r_mutex);
    V(read_try);

    read_database();

    P(r_mutex);
    read_count--;
    if (read_count == 0) V(rw_mutex);
    V(r_mutex);
}

```

考试层次

408 常见题主要要求读者优先、写者优先或给定策略下填 PV。解题时先确定题目要求的**公平策略**，再决定是否需要额外闸门；不要把某一版模板当成唯一“读者-写者答案”。

6 模式三：理发师——有限准入 + 双向会合

6.1 模型抽象

理发店有 M 个等待座位。顾客到达：有空位则等待，无空位则离开；理发师无顾客时睡眠。
这个题可以理解为：

- 顾客产生服务请求；
- 理发师消费服务请求；
- 等待椅是有限容量；
- 顾客与理发师还需完成一次 rendezvous（会合）。

6.2 经典信号量

信号量 / 共享变量	初值 (Initial)	含义与作用
customers	0	正在等待服务的顾客数量（同步信号量）
barbers	0	已准备好接待顾客的理发师许可（同步信号量 / 双向会合）
mutex	1	保护 waiting_customers 计数器的互斥信号量
waiting_customers	0	当前在等待椅子上坐着的顾客人数（共享整型变量）

```
barber() {
    while (true) {
        P(customers);
        P(mutex);
        waiting_customers--;
        V(barbers);
        V(mutex);
        cut_hair();
    }
}

customer() {
    P(mutex);
    if (waiting_customers < CHAIRS) {
        waiting_customers++;
        V(customers);
        V(mutex);
        P(barbers);
        get_haircut();
    } else {
        V(mutex);
        leave_shop();
    }
}
```

6.3 它为什么比生产者-消费者多一层

生产者-消费者只需确保“产品存在”；理发师问题还要让一个具体顾客知道“已有理发师可以服务”，因此多了 barbers 这一会合信号。题目若增加多名理发师、不同服务等级或排队规则，就会进一步引入身

份匹配与公平性。

7 模式四：哲学家进餐——多资源获取 + 循环等待

7.1 真正训练的是资源依赖图

最朴素方案：每人先左后右。若所有哲学家都拿到左筷子：

$$P_0 \rightarrow C_1 \rightarrow P_1 \rightarrow C_2 \rightarrow \cdots \rightarrow P_{N-1} \rightarrow C_0 \rightarrow P_0$$

形成循环等待。

7.2 常见死锁规避思路

1. 限制准入：最多允许 $N - 1$ 人同时尝试拿筷子；
2. 资源全序：所有人按统一编号顺序获取资源，破坏循环等待；
3. 原子化资格判定：在受保护状态中判断左右邻居，只有同时满足才进入 EATING；否则阻塞等待通知。

7.3 状态数组 + 私有信号量的阻塞范式

```
semaphore mutex = 1;
semaphore self[N] = {0};
state[N] = {THINKING};

test(i) {
    left = (i + N - 1) % N;
    right = (i + 1) % N;
    if (state[i] == HUNGRY &&
        state[left] != EATING &&
        state[right] != EATING) {
        state[i] = EATING;
        V(self[i]);
    }
}

take_forks(i) {
    P(mutex);
    state[i] = HUNGRY;
    test(i);
    V(mutex);
    P(self[i]);
}

put_forks(i) {
    P(mutex);
```

```

    state[i] = THINKING;
    test(left(i));
    test(right(i));
    V(mutex);
}

```

这里 `mutex` 保护的是状态判定过程，`self[i]` 表达“哲学家 i 何时获得进餐资格”。

不要过度声称

这种状态判定方案可以设计为**无死锁**，但“绝对无饥饿”还依赖唤醒/调度公平性和具体判定策略。考试若只问死锁避免，不必额外假设强公平性。

8 模式五：吸烟者——条件分派与精确通知

8.1 核心不是“材料”，而是多路条件

供应者每轮制造不同条件，只有匹配的一个吸烟者应继续：

$$Condition_0 \rightarrow Smoker_0, \quad Condition_1 \rightarrow Smoker_1, \quad Condition_2 \rightarrow Smoker_2.$$

最直接的 PV 思路是：为每种可区分事件设置独立同步信号量。

8.2 简洁 408 模板

设 `agent=1` 表示桌面空、允许供应下一组；`offer[3]=0` 表示三种匹配事件尚未发生。

```

provider() {
    while (true) {
        P(agent);
        choose k in {0,1,2};
        put_materials_for(k);
        V(offer[k]);
    }
}

smoker(k) {
    while (true) {
        P(offer[k]);
        take_materials();
        make_and_smoke();
        V(agent);
    }
}

```

这体现：

复杂条件同步 → 把不同条件编码为不同等待队列/信号量

关于“代理人模式”

增加 Agent/检查者可以把匹配分发移入中间层，但不是 408 必需模板。若多个代理共同消费同一检查信号，必须防止错误消费事件或形成无效唤醒循环；考试中优先为每类条件设置独立信号量。

9 模式六：前驱图——把 Happens-Before 边编码成事件信号量

9.1 图模型

若有有向边：

$A \rightarrow C$

表示 A 完成是 C 开始的前置事实。若两操作位于不同执行流，就需要显式同步；若在同一线程中且程序顺序已保证，则通常无需额外信号量。

一条跨线程前驱边

为跨线程边 $A \rightarrow C$ 定义初值为 0 的信号量 S_{AC} ：

$A; V(S_{AC}) \qquad P(S_{AC}); C$

初值为 0 正是因为“A 已完成”这一事实在系统初始时尚不存在。

9.2 2022 风格示例

依赖：

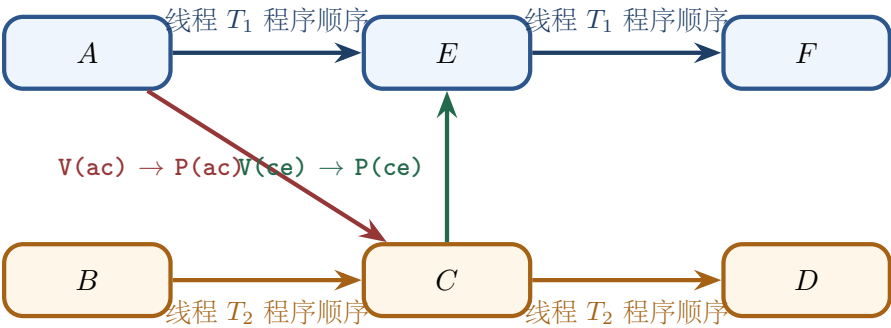
$A \rightarrow C, B \rightarrow C, C \rightarrow D, C \rightarrow E, E \rightarrow F$

线程：

$T_1 = \{A, E, F\}, \quad T_2 = \{B, C, D\}.$

只保留跨线程边：

$A \rightarrow C, \quad C \rightarrow E.$



于是：

```
semaphore ac = 0, ce = 0;

T1() {
```

```

    A();
    V(ac);
    P(ce);
    E();
    F();
}

T2() {
    B();
    P(ac);
    C();
    V(ce);
    D();
}

```

9.3 新增模式：Barrier 与 fork/join

Fork/Join 线程 A 创建工作线程 B，A 必须等待 B 完成：这就是一次性事件。

$$S_{done} = 0, \quad B : V(S_{done}), \quad A : P(S_{done}).$$

Barrier 若 N 个线程必须全部完成阶段 1 后才能进入阶段 2，核心是不变量：

$$\text{phase2 begins} \Rightarrow \text{count} = N.$$

简单一次性 barrier 可用受保护计数器 + gate 信号量构造；可复用 barrier 则需要防止下一轮线程混入上一轮，属于更高阶变式。

10 从 PV 到管程：wait 为什么必须“释放并睡眠”

10.1 问题不是“wait 会不会释放锁”，而是不能丢失唤醒

设线程 T_1 检查：

$$\text{count} = 0$$

准备等待。如果“释放锁”和“真正进入等待队列”之间存在可被其他线程插入的空窗：

```

T1: check count == 0
      <- context switch
T2: count = 1
T2: signal()
T1: sleep

```

那么通知发生时 T_1 还没睡，之后却永久睡下，形成 **lost wakeup**。

因此条件等待必须具有语义：

atomically release mutex and block

被唤醒后，又必须：

reacquire mutex before wait returns

这样线程重新检查共享状态时仍处于受保护环境。

10.2 标准思维模板

```
lock(m);
while (!predicate)
    wait(cv, m);    // 原子释放 m + 阻塞；返回前重新获得 m
use_state();
unlock(m);
```

修改方：

```
lock(m);
change_state();
signal(cv);
unlock(m);
```

两个必须纠正的直觉

- `signal()` 并不等于“条件一定成立且被唤醒者立即运行”；它通常只是使等待者有机会变为可运行状态。
- 条件变量本身不是条件；真正的条件是共享状态上的布尔谓词，所以等待者醒来后必须用 `while` 重检。

10.3 管程是什么

管程可以概括为：

Shared State + Operations + Implicit Mutual Exclusion + Condition Variables

它的核心价值是把“共享状态及其合法同步协议”封装在一个模块中，减少调用者随意破坏不变量的机会。

11 死锁：PV 题的第二条主线

11.1 从等待图而不是代码表面判断

若 T_i 等待 T_j 才能释放/制造的资源或条件，可画：

$$T_i \rightarrow T_j.$$

若形成闭环：

$$T_1 \rightarrow T_2 \rightarrow \cdots \rightarrow T_k \rightarrow T_1$$

就应高度警惕死锁。

11.2 Coffman 四条件的角色

经典必要条件：互斥、请求并保持、不可剥夺、循环等待。学习时不要只背名字，而要问：**我的设计破坏了哪一条？**

死锁规避/预防策略	典型工程/算法思路与实现手段
破坏请求并保持	一次性原子申请全部资源；若有任意一个资源不可用则释放已拿到的资源
破坏循环等待	给系统所有资源类型建立全局唯一编号（全序），强制按递增顺序申请
限制并发准入	哲学家进餐中限制最多允许 $N - 1$ 人同时尝试获取资源
避免不安全状态	动态分配前运行银行家算法 (Banker's Algorithm)，拒绝进入不安全状态
检测后恢复机制	允许死锁发生，后台定期运行等待环检测算法，通过剥夺/撤销任务进行恢复

11.3 Deadlock / Livelock / Starvation

异常并发现象	本质驱动原因与系统表现
死锁 (Deadlock)	多个执行流因相互等待对方持有的锁/许可而全部阻塞，整个系统无法向前推进
活锁 (Livelock)	执行流持续改变状态/重试（未阻塞），但由于缺乏协调导致整体目标无法推进
饥饿 (Starvation)	系统整体有推进（无死锁），但某个特定执行流因调度不公长期拿不到资源

12 经典 PV 题不要背答案：把它们看成“并发结构模式库”

经典同步题型	真正抽象结构	解题第一反应与建模板块
生产者-消费者	容量计数 + 共享结构互斥	写 $0 \leq count \leq N$ ，寻找 empty / full / mutex
读者-写者	群体准入 + 元数据保护 + 公平策略	谁可并行？谁必须独占？谁记录群体状态？
哲学家进餐	多资源同时需求 + 循环等待	画资源依赖图；考虑全序、准入限制或原子资格判断
吸烟者问题	多路条件分派 / Selective Wakeup	不同条件是否应有不同事件信号量？
理发师问题	有限准入 + 服务/请求双向会合	分清等待容量、请求数量、服务准备三类状态
前驱图 (DAG)	Happens-Before 显式前驱边	只为跨执行流依赖建立初值为 0 的事件信号量
Barrier 屏障	阶段协同同步	所有人到齐之前无人能越过阶段边界
Fork / Join	一次性异步完成事件	完成者执行 V，等待者执行 P，初值设为 0

13 408 大题作答模板：从自然语言到 PV 伪代码

13.1 第一遍：只翻译业务，不写 P/V

先写：

```
producer: produce -> put
consumer: get -> consume
```

或者：

```
reader: enter_read -> read -> leave_read
writer: enter_write -> write -> leave_write
```

避免一边读题一边随手插 P/V，导致局部正确、整体矛盾。

13.2 第二遍：建立状态表

建议草稿固定写：

对象 / 资源	初始状态与许可数量	谁会改变/消费该许可?
共享数据实体	例如 <code>buffer / count</code>	生产者 / 消费者（通过临界区保护）
互斥许可	<code>Initial = 1</code>	进入/离开临界区的执行流成对 P/V
资源许可	<code>Initial = N</code> 或 <code>0</code>	生产方 V 增加，消费方 P 减少
事件许可	<code>Initial = 0</code>	前驱动作完成者 V，后继等待者 P

13.3 第三遍：在“等待点”和“制造条件点”成对放置 P/V

每一个 P 都应能回答：

我现在具体缺少哪一份事实/资源/许可？谁能够在未来执行 V 制造它？

每一个 V 都应能回答：

我刚刚完成了什么状态变化，使哪类等待者现在可能继续？

13.4 第四遍：四类证明

- 1. **Safety**：临界区是否互斥？容量是否越界？前驱是否被违反？
- 2. **Deadlock**：有没有“持有 A 等 B，而制造 B 的人必须先拿 A”？有没有资源环？
- 3. **Liveness**：每个阻塞点未来是否存在可达的 V？
- 4. **Performance**：耗时业务是否被不必要地放进互斥区？是否人为串行化？

14 高频错因诊断表

错误现象	根本认知缺口	修复与排查提问
<code>mutex</code> 初值乱设	没理解“一个互斥进入许可”	系统初始能允许几人同时进入？
<code>full/empty</code> 写反	没从初始物理状态推许可	一开始已有多少产品？多少空位？
看到共享变量就全加锁	没区分共享状态与不变量	哪些多步更新真的不能交错？
锁包住生产/计算耗时操作	不理解临界区粒度	哪部分只是私有计算？

错误现象	根本认知缺口	修复与排查提问
先拿 <code>mutex</code> 后等待容量死锁	没画等待依赖	制造我等待条件的人是否也需要我持有的锁?
读者写者漏保护 <code>count</code>	忽视元数据也是共享状态	谁在决定“第一个/最后一个”?
<code>V</code> 了错误信号量	没把 <code>V</code> 当成“制造事实”	我刚刚制造了哪一种可消费条件?
前驱图边全设信号量	没区分线程内部程序顺序	这条依赖是否已经由同一线程顺序保证?
哲学家只背左右筷顺序	没看到资源等待环	是否存在全序或原子获取策略?
把唤醒当作立即运行	混淆 <code>Blocked→Ready</code> 与 <code>Ready→Running</code>	谁还必须经过 <code>scheduler</code> ?
把 <code>semaphore</code> 当 <code>mutex</code>	混淆许可与所有权	这里管理的是“谁拥有锁”还是“还有几份许可”?

15 考场速查：PV 十问

拿到 PV 大题，按顺序问

1. 有几类执行流?

2. 它们共享哪些状态/资源?

3. 每个共享状态的初值是什么?

4. 哪些状态更新必须互斥?

5. 哪些动作必须等待某个条件/前驱?

6. 每个等待条件在现实中对应什么“许可”? 初值多少?

7. 谁在什么动作完成后制造这个许可 (V)?

8. 是否持锁等待了只能由竞争者制造的条件?

9. 是否存在循环等待、饥饿或无意义的大临界区?

10. 最终伪代码是否保持所有不变量，并且每个阻塞点都有未来可达的唤醒路径?

16 专题总结：把 PV 从代码模板提升为状态约束与语义建模

最终核心模型

Flows → Shared State → Interleavings → Invariant → Constraints

→ Semaphore/PV → Progress → Cost

不要看到生产者-消费者就机械写 `empty/full/mutex`。按以下顺序建模：

1. 先找到谁在等待什么事实；

2. 再找到**谁能够制造这个事实**；
3. 用信号量把事实变成可被原子消费/生产的许可；
4. 用互斥保护共享状态的不变量；
5. 最后证明所有允许交错下系统既不会进入坏状态，也不会因为等待环而无法推进。

与进程调度的接口

P 操作可能使执行流从 Running 进入 Blocked，V 操作可能使等待者从 Blocked 进入 Ready；Ready→Running 仍由调度器决定。PV 只表达许可和等待，不直接分配 CPU。

边界检查：四句不能绝对化

四句不应绝对化的话

1. “所有 PV 题都必须先同步 P 后互斥 P”——标准有界缓冲区正确，但更一般原则是避免持锁等待依赖环。
2. “V 的顺序永远不重要”——简单模板里可能不影响 Safety，但会影响唤醒竞争、性能，复杂协议还可能影响语义。
3. “二元信号量就是 mutex”——考试可用于互斥，但严格语义中 mutex 有所有权，semaphore 表示许可。
4. “某个哲学家解法一定无饥饿”——无死锁不自动等于公平；饥饿还取决于调度和唤醒策略。